Grocery List Manager

INTRODUCTION:

The Grocery List Manager is a full-stack web application developed to help users efficiently manage their daily grocery shopping needs. Traditional methods such as writing lists on paper or remembering items mentally are often inconvenient and prone to errors. This application provides a digital solution that allows users to create, update, and track their grocery lists easily.

The system is built using modern web technologies following the MERN (MongoDB, Express.js, React.js, Node.js) stack. It provides a user-friendly interface and secure authentication to ensure personalized access.

The application allows users to:

- Add grocery items
- Edit or delete items
- Mark items as purchased
- Organize lists efficiently

This system improves convenience, reduces time, and ensures better organization of daily shopping tasks.

SCENARIO BASED CASE STUDY

To understand the practical use of the Grocery List Manager, consider the following scenario:

User Perspective:

Maneesh, a college student living in a hostel, frequently forgets items while shopping. By using the Grocery List Manager, he logs into the application and creates a list of items such as milk, vegetables, and snacks. While shopping, he marks items as completed in real-time. This ensures he does not miss anything and saves time.

SYSTEM REQUIREMENTS:

Software Requirements:

To develop and run the Grocery List Manager application smoothly, the following software tools and technologies are required. These ensure proper functioning of both frontend and backend operations.

1. Development Environment:

- **Operating System:** Windows, macOS, or Linux – supports cross-platform development.
- **Node.js:** Provides runtime environment for backend development.
- **npm (Node Package Manager):** Used to install and manage project dependencies.

2. Frontend Technologies:

Frontend technologies are used to design and build the user interface of the application.

- **React.js:** Used for creating interactive and dynamic UI components.
- **Web Browser (Chrome/Firefox):** Used to run and test the application.

3. Backend Technologies:

Backend technologies handle server-side logic and data processing.

- **Node.js:** Used for executing JavaScript on the server side.
- **Express.js:** Helps in building APIs and handling routing efficiently

4. Database:

The database is used to store and manage application data.

- **MongoDB Atlas:** Cloud-based NoSQL database for storing user data and grocery items.

5. Development and Testing Tools:

These tools help in writing, testing, and debugging the application.

- **Visual Studio Code:** Code editor used for development.
- **Postman:** Used to test API requests and responses.

Hardware Requirements

The Grocery List Manager application does not require high-end hardware and can run efficiently on standard computing systems. The following hardware components are sufficient for smooth development and execution of the application.

- **Processor:**A processor with a minimum speed of 1 GHz or higher is required. A faster processor improves application performance and reduces response time during development and execution.

- **Memory (RAM):**The system should have at least **2 GB RAM**, while **4 GB or more is recommended** for better performance, especially when running both frontend and backend simultaneously.

- **Storage:**A minimum of 500 MB of free disk space is required to install the necessary tools, dependencies, and project files. Additional space may be needed for updates and database storage.

- **Display:**A display with a resolution of at least **1366 × 768 pixels** is recommended to ensure proper visibility of the user interface and smooth interaction.

- **Internet Connection:**A stable internet connection is required for accessing cloud services like MongoDB Atlas, installing dependencies, and running the application efficiently.

PROJECT ARCHITECTURE:

The Grocery List Manager application follows a **three-tier architecture**, which separates the system into different layers for better organization, scalability, and maintainability. Each layer performs a specific function and interacts with the others to process user requests efficiently.

- **Presentation Layer (Frontend):**The presentation layer is developed using **React.js** and **Tailwind CSS**. It is responsible for providing the user interface through which users interact with the system.

  This layer includes pages such as login, dashboard, and grocery list management. It collects user inputs (like adding items or creating lists) and sends requests to the backend. It also displays the data received from the server in a structured and user-friendly format.

- **Application Layer (Backend):**The application layer is built using **Node.js** and **Express.js**. It handles all the business logic and processes user requests received from the frontend.

  This layer manages functionalities such as user authentication, list creation, item management, and updating purchase status. It receives requests from the frontend, processes them, and communicates with the database to store or retrieve data.

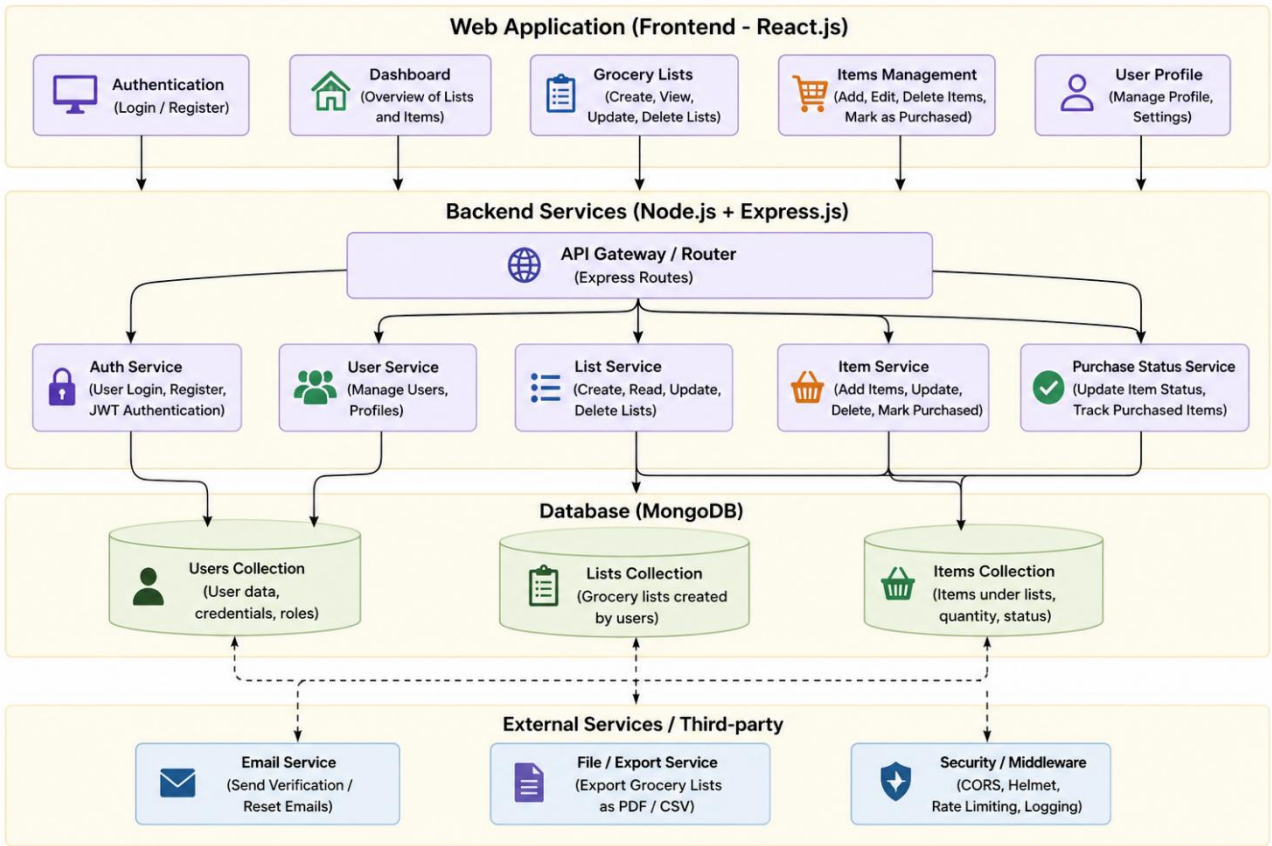
- **Data Layer (Database):**The data layer uses **MongoDB** to store all application data. It maintains collections such as users, lists, and grocery items.

  MongoDB stores data in a flexible format, making it easy to manage and update. The backend interacts with the database using Mongoose to perform operations like inserting, updating, and deleting data.

- **Data Flow:**
  The overall data flow in the system is as follows:

  1. The user interacts with the frontend (React UI)
  2. The frontend sends a request to the backend via API
  3. The backend processes the request using Express
  4. The backend communicates with MongoDB to store/retrieve data
  5. The response is sent back to the frontend
  6. The frontend updates the UI according

TECHNICAL ARCHITECTURE:



PROJECT ARCHITECTURE:

The Grocery List Manager application is developed using the **MERN stack**, which consists of **React.js** for the frontend, **Node.js and Express.js** for the backend, and **MongoDB** as the database. The system follows a **client-server architecture**, where the frontend and backend communicate through RESTful APIs.

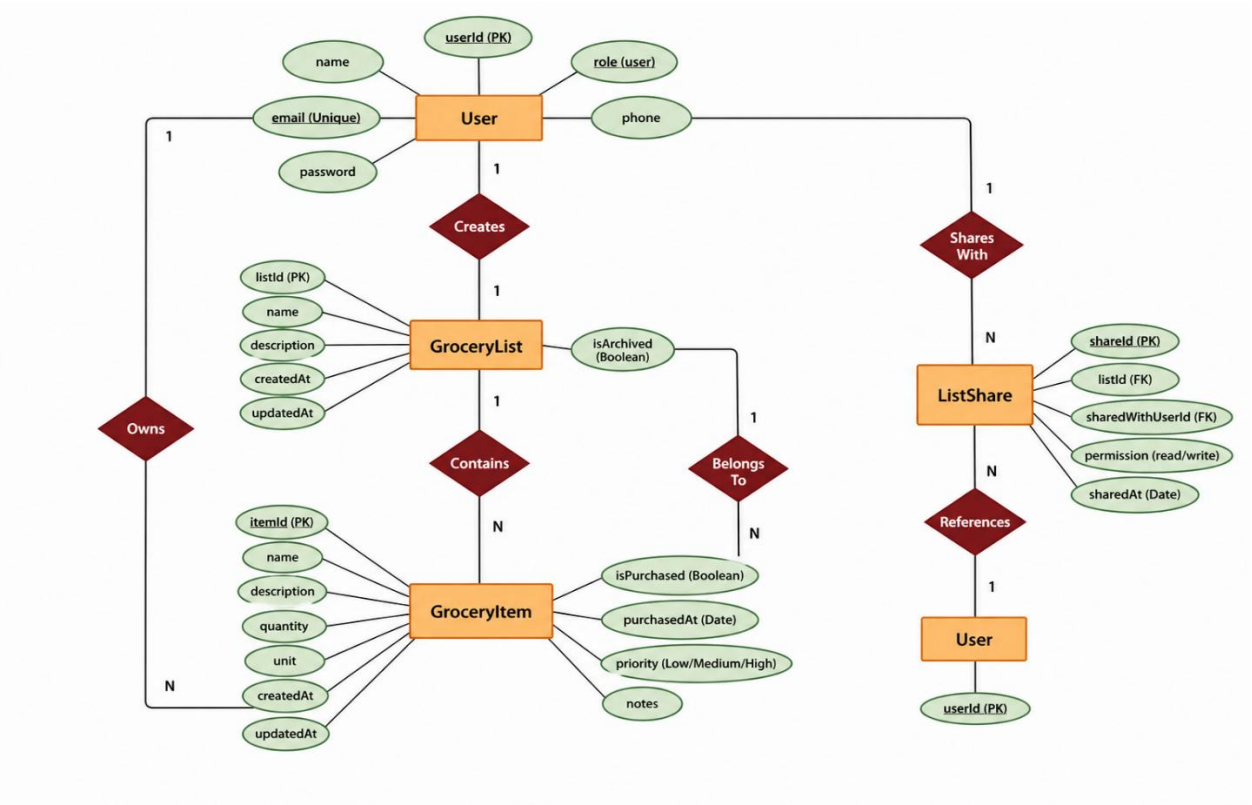
The **frontend (client side)** is responsible for providing an interactive user interface. It collects user inputs such as creating grocery lists, adding items, and updating purchase status. The frontend sends HTTP requests (GET, POST, PUT, DELETE) to the backend and dynamically displays the responses received from the server.

The **backend (server side)** is built using Node.js and Express.js and acts as the core of the application. It handles all business logic, including user authentication, list management, and item operations. The backend processes incoming requests, performs necessary validations, and returns appropriate responses to the frontend.

The backend communicates with the **MongoDB database** using **Mongoose**, which provides schema-based data modeling. It is used to store and manage data such as users, grocery lists, and items. Operations like creating, updating, retrieving, and deleting data (CRUD operations) are efficiently handled through this layer.

ER DIAGRAM:

The Entity-Relationship (ER) diagram of the Grocery List Manager represents the structure of the database and the relationships between different entities in the system. It shows how users, lists, and grocery items are connected and how data is organized for efficient storage and retrieval.



ENTITY-RELATIONSHIP (ER) MODEL DESCRIPTION:

1. Entities:

• User:
Represents the users of the system. It includes attributes such as *userId, name, email, and password*. Each user can create and manage multiple grocery lists.

• List:
Represents a grocery list created by a user. It includes attributes such as *listId, listName, and createdAt*. Each list is associated with a specific user.

• Item:
Represents individual grocery items within a list. It includes attributes such as *itemId, itemName, category, quantity, price, and purchaseStatus*. Each item belongs to a particular list.

2. Relationships:

• User–List Relationship:
A user can create multiple grocery lists, but each list belongs to only one user. This represents a one-to-many (1:N) relationship.

• List–Item Relationship:
A list can contain multiple items, but each item belongs to only one list. This also represents a one-to-many (1:N) relationship

.

3. Attributes:

Each entity contains specific attributes that define its properties:

- The User entity stores personal and authentication details.
- The List entity stores information about grocery lists created by users.
- The Item entity stores detailed product information such as quantity, category, and purchase status.

4. Primary Keys:

Each entity is uniquely identified using a primary key:

- User → userId
- List → listId
- Item → itemId

5. Foreign Keys:

Foreign keys are used to establish relationships between entities:

- The List entity contains *userId* as a foreign key to link it with the User.
- The Item entity contains *listId* as a foreign key to link it with the List.

This ER model clearly represents the structure of the database and ensures proper organization of data, maintaining relationships between users, lists, and items efficiently.

Overall Description:

The ER diagram clearly shows how users manage multiple grocery lists and how each list contains several items. This structure ensures proper organization of data, avoids redundancy, and allows efficient retrieval and updates of grocery information.

KEY FEATURES:

The Grocery List Manager application is designed to provide a simple and efficient way to manage daily grocery tasks. The key features of the system are as follows:

- **User Authentication:**

  The system provides a secure login and registration process, ensuring that each user has a personalized account and access to their own data.

- **Grocery List Management:**

  Users can create multiple grocery lists with custom names. They can organize their shopping items based on different needs such as monthly or dinner party shopping.

- **Item Management:**

  Users can add items to a list with details such as product name, category, quantity, and price and even reference image . Items can also be edited or deleted easily.

- **Purchase Tracking:**

  The application allows users to mark items as purchased or unpurchased, helping them track completed and pending items.

- **Search Functionality:**

  Users can search for their grocery lists by name, making it easier to find and manage lists quickly.

- **Total Cost Calculation:**

  The system automatically calculates the total cost of items in a list, helping users manage their budget effectively.

- **Image Support:**

  Users can add an image for each product using an image URL, making the list more visual and user-friendly.

- **Excel Export Feature:**

  The application allows users to export their grocery list data into an Excel file for offline use and record keeping.

- **Responsive User Interface:**

  The application is designed with a clean and responsive UI, ensuring smooth usage across different devices such as desktops and mobiles.

ROLES AND RESPONSIBILITIES:

In the Grocery List Manager application, the primary role is the **User**, who interacts with the system to manage grocery lists and items efficiently. The responsibilities are as follows:

1. user:

- **Account Management:**
  The user is responsible for creating an account and logging into the system securely. They must provide valid credentials and ensure the confidentiality of their login details.
- **Grocery List Management:**
  The user can create, view, search, and delete multiple grocery lists. Each list can be organized based on different shopping needs.
- **Item Management:**
  The user can add items to a list with details such as product name, category, quantity, price, and image. They can also update or delete items as required.
- **Purchase Tracking:**
  The user can mark items as purchased or unpurchased. This helps in tracking which items are completed and which are pending.
- **Expense Monitoring:**
  The user can view the total cost of items in a list, helping them manage their budget.

- **Data Responsibility:**
  The user is responsible for maintaining accurate data in the application and regularly updating their grocery lists.

2. System (Application):

- **System (Application):**
  The system stores user data, grocery lists, and items securely using MongoDB.
- **Data Processing:**
  The system processes user requests such as adding items, updating lists, and calculating total cost.
- **Security:**
  The system ensures secure authentication and protects user data from unauthorized access.

User Flow:

The user flow of the Grocery List Manager application describes how a user interacts with the system to perform different operations. The process is simple and user-friendly.

1. User Login Flow:

- The user opens the application
- Enters email and password
- The system verifies the credentials
- If valid, the user is redirected to the dashboards.

2. Grocery List Flow:

- The user creates a new grocery list by entering a list name
- The created lists are displayed on the “My Lists” page
- The user can search for lists using the search bar
- The user can delete any list if not neededthreshold.

3. Item Management Flow:

- The user opens a specific grocery list
- Adds items by entering details such as name, category, quantity, price, and image
- The added items are displayed in the list

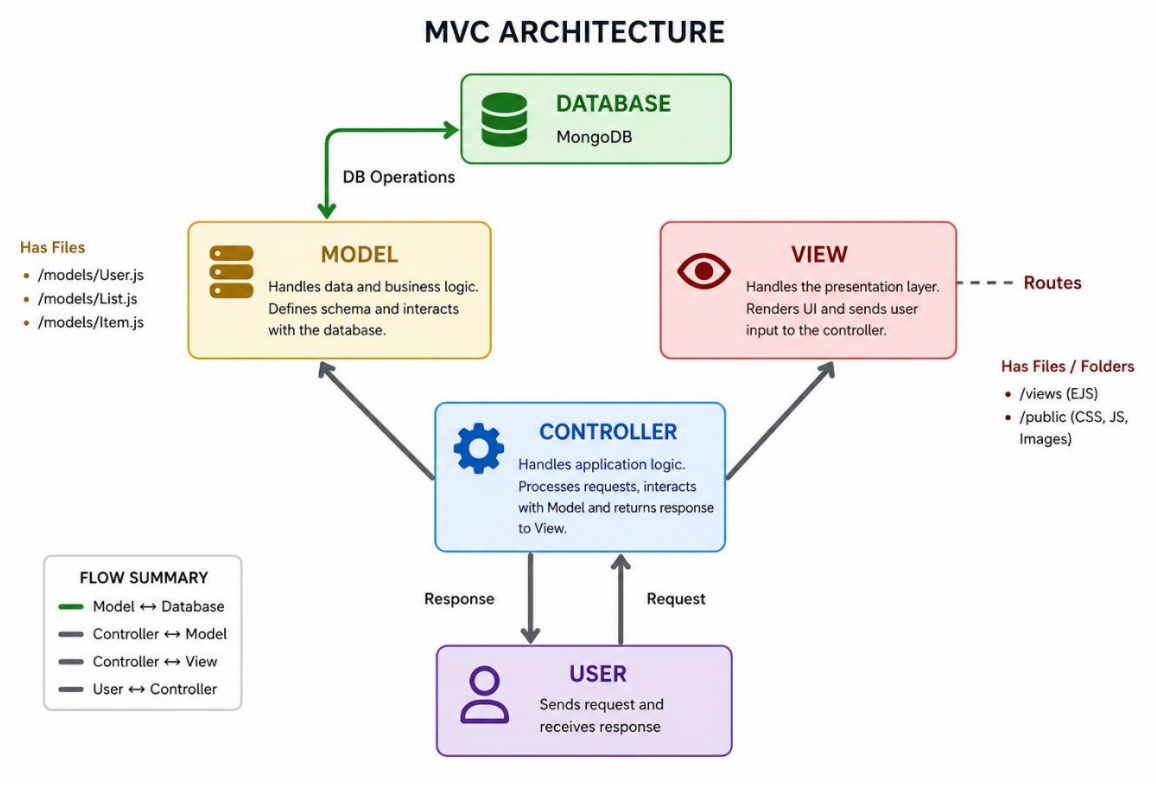
4. Purchase Tracking Flow:

- The user marks items as purchased using a checkbox
- The system updates the item status
- Purchased and pending items can be viewed separately

5. Export Flow:

- The user selects the export option and download a excel sheet of the groceries list.

MVC PATTERN:



MVC ARCHITECTURE:

The Grocery List Manager application follows the **Model-View-Controller (MVC) architecture**, which divides the system into three main components: Model, View, and Controller. This separation enhances code organization, maintainability, and scalability.

The **Model** represents the data layer and handles all database-related operations. It defines the structure of the data using schemas and interacts with MongoDB through Mongoose.
The **View** represents the presentation layer, implemented using React.js. It is responsible for displaying data to the user and capturing user inputs through the interface.

The **Controller** acts as the intermediary between the Model and the View. It processes user requests, applies business logic, interacts with the Model to perform operations, and sends appropriate responses back to the View.
By following the MVC pattern, the application ensures a clear separation of concerns, allowing independent development and modification of each component. This results in a more organized, scalable, and easy-to-maintain system.

Model Layer (Data Layer):

The Model layer is responsible for managing the data of the application. It defines the structure of the database and handles operations such as creating, updating, retrieving, and deleting data.

In this project, MongoDB is used as the database, and Mongoose is used to define schemas. The main models include User, List, and Item, which store user details, grocery lists, and item information respectively.

.

View Layer (User interface):

The View layer represents the frontend of the application. It is developed using React.js and is responsible for displaying data to the user.

This layer includes pages such as login, dashboard, and grocery list interface. It collects user input and sends requests to the controller through API calls.

Controller Layer:

The Controller layer acts as a bridge between the Model and View layers. It handles user requests, processes input data, and performs the required operations.

In this project, controllers are implemented using Express.js. They manage functionalities such as user authentication, list creation, item management, and updating purchase status.

Advantages of MVC in This Project:

- Separation of concerns improves code structure.
- Easy to maintain and update.
- Allows scalability for future enhancements.
- Makes debugging and testing easier.
- Supports parallel development of frontend and backend.

Project Setup And Configuration:

The Grocery List Manager project is developed using the MERN stack, which includes separate setup for the frontend and backend. The project is organized into two main folders to manage the application efficiently.

Creating Project Folder:

1. Create a main project folder with the name *Grocery List Manager*.
2. Inside the folder, create two subfolders:

    - **frontend** (for React application)
    - **backend** (for Node.js and Express server)

3. Open the project folder in Visual Studio Code for development.
4.

Client Setup (Frontend Initialization):

1. Open the **frontend** folder in the terminal.
2. Initialize the React application using Vite.
3. Select:

    - Framework: React
    - Variant: JavaScript

4. Install dependencies using npm.
5. Start the development server to verify setup.
6. Install required packages such as:

    - Axios (for API calls)
    - React Router (for navigation)

7. Organize the project structure by creating folders:

    - components
    - pages
    - services

8. Begin developing UI components such as login page, dashboard, and list page.

Server Setup (Backend Initialization):

1. Open the **backend** folder in the terminal.
2. Initialize the backend project using npm.
3. Install required packages:

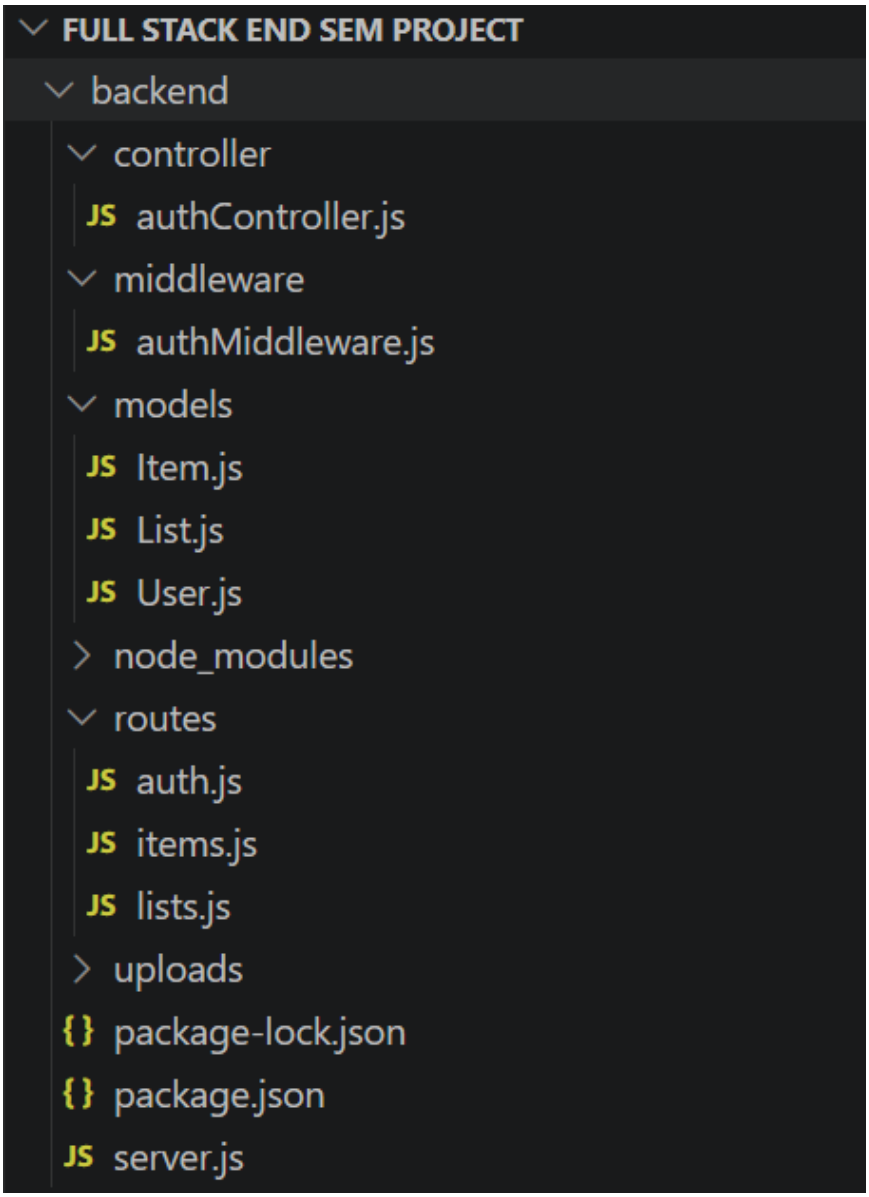
    - Express.js
    - Mongoose
    - CORS
    - dotenv
    - jsonwebtoken (JWT)
    - bcrypt

4. Create the main file (**server.js**) as the entry point.
5. Organize backend structure by creating folders:

    - models, controller, routes, middleware

BACKEND DEVELOPMENT:

Backend Structure:



CONTROLLERS

• authController.js → Handles user authentication operations such as registration and login. It manages user credentials, validates input, and generates tokens for secure access.

```
    user = new User({
      username: normalizedUsername,
      email: normalizedEmail,
      password: hashedPassword
    });

    await user.save();

    const token = jwt.sign({ id: user._id }, JWT_SECRET, { expiresIn: '1d' });
    res.status(201).json({ token, user: { id: user._id, username: user.username, email: user.email } });
  } catch (err) {
    console.error('Registration Error:', err);
    if (err.name === 'ValidationError') {
      const firstError = Object.values(err.errors)[0];
      return res.status(400).json({ message: firstError.message });
    }
    res.status(500).json({ message: 'Server error: ' + err.message });
  }
};
const login = async (req, res) => {
  const { email, password } = req.body;
  try {
    const user = await User.findOne({ email });
    if (!user) return res.status(400).json({ message: 'Invalid credentials' });

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(400).json({ message: 'Invalid credentials' });
```

MIDDLEWARE

• authMiddleware.js → Handles authentication and authorization using JWT tokens. It protects routes and ensures that only authenticated users can access specific functionalities like creating lists or adding items.

```
const jwt = require('jsonwebtoken');

const authMiddleware = (req, res, next) => {
  const token = req.header('Authorization');

  if (!token) {
    return res.status(401).json({ message: 'No token, authorization denied' });
  }

  try {
    const decoded = jwt.verify(token.replace('Bearer ', ''), process.env.JWT_SECRET || 'supersecretjwt');
    req.user = decoded; // { id, iat, exp }
    next();
  } catch (err) {
    res.status(401).json({ message: 'Token is not valid' });
  }
};

module.exports = authMiddleware;
```

MODELS

• User.js → Defines the schema for storing user details such as name, email, and password. It is used for authentication and user management.
• List.js → Defines the schema for storing grocery lists created by users. It includes fields like list name and userId to associate lists with users.
• Item.js → Defines the schema for storing grocery items within a list. It includes attributes such as item name, category, quantity, price, and purchase status.

```
const itemSchema = new mongoose.Schema({
  listId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'List',
    required: true
  },
  category: {
    type: String,
    enum: ['Fruits', 'Vegetables', 'Dairy', 'House Essentials', 'Others'],
    required: true
  },
  name: {
    type: String,
    required: true,
    trim: true
  },
  price: {
    type: Number,
    required: true,
    min: 0
  },
  quantityType: {
    type: String,
    enum: ['Kg/Gram', 'Liter', 'Pieces'],
    required: true
```

```
const mongoose = require('mongoose');

const listSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true
  },
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  }
}, { timestamps: true });

module.exports = mongoose.model('List', listSchema);
```

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    trim: true,
    minlength: 3
  },
  email: {
    type: String,
    required: true,
    unique: true,
    trim: true,
    lowercase: true
},
  password: {
    type: String,
    required: true,
    minlength: 6
  }
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

ROUTES

• auth.js → Contains API routes for user authentication such as registration and login.
• lists.js → Contains API routes for managing grocery lists, including creating, retrieving, and deleting lists.
• items.js → Contains API routes for managing items within a list, such as adding, updating, deleting, and marking items as purchased.

SERVER CONFIGURATION

• server.js → Main entry point of the backend application. It sets up the Express server, connects to MongoDB, configures middleware, and defines API routes for the application.

```
const express = require('express');
const { register, login, getMe } = require('../controller/authController');

const router = express.Router();

const auth = require('../middleware/authMiddleware');

router.post('/register', register);
router.post('/login', login);
router.get('/me', auth, getMe);

module.exports = router;
```

DATABASE DEVELOPMENT:

DATABASE SETUP (MONGODB CONFIGURATION)

1. Mongoose is installed to interact with the MongoDB database and define schemas for the application.
2. The database connection is configured in the backend (server.js) to manage connectivity.
3. MongoDB Atlas is used as the cloud database, allowing secure and remote data storage.
4. Environment variables are configured using dotenv to securely store sensitive data such as the database connection URI.

DATABASE CONNECTION

The application establishes a connection to MongoDB using Mongoose before performing any database operations. This ensures reliable communication between the backend and the database.

SCHEMA DESIGN

The Grocery List Manager application uses three main schemas to organize and manage data efficiently:

1. User Schema:

This schema stores user-related information such as userId, name, email, and password. It is used for authentication and managing user accounts. Each user can create and manage multiple grocery lists.

2. List Schema:

This schema stores details of grocery lists created by users. It includes attributes such as listId, listName, and userId. The userId acts as a reference to associate each list with a specific user.

3. Item Schema:

This schema stores information about individual grocery items within a list. It includes attributes such as itemId, itemName, category, quantity, price, and purchaseStatus. Each item is linked to a specific list using listId.

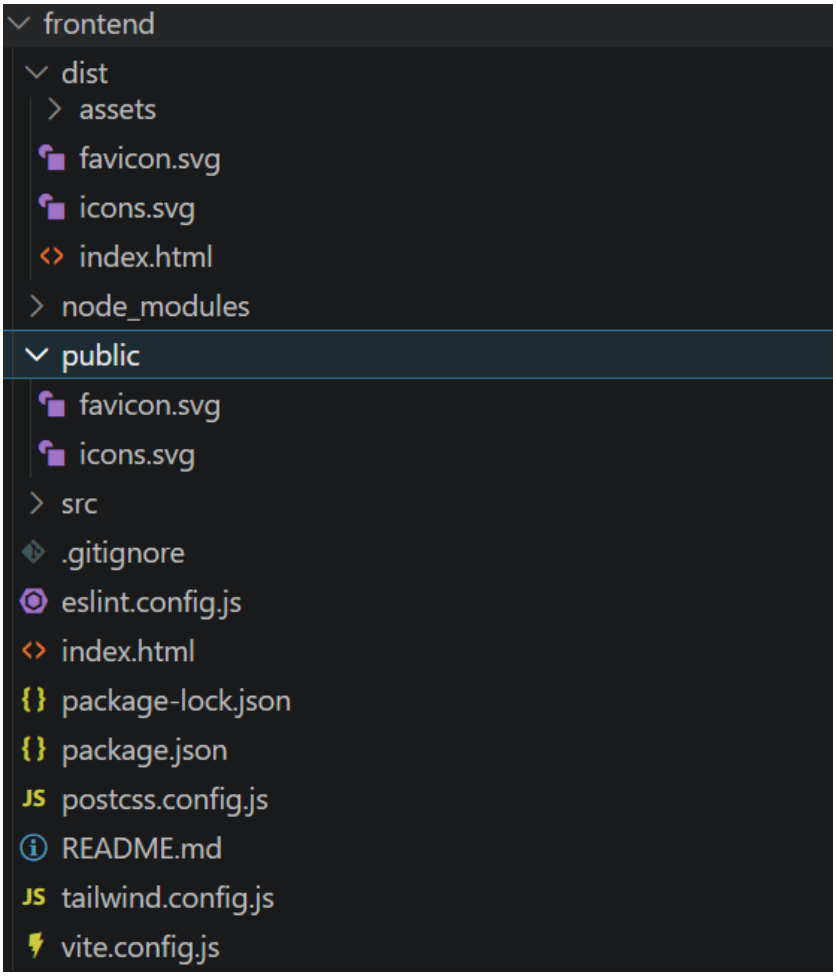
This schema design ensures efficient data organization and maintains relationships between users, lists, and items, enabling smooth data management and retrieval in the application.

Backend Explanation video:

https://drive.google.com/file/d/1N74xUSej7Xv1NGPFgHrQNTzke4FXm0XY/view?usp=drive_link

FRONT-END DEVELOPMENT:

Frontend Structure:



FRONTEND COMPONENTS (USER SIDE)

• **Login.jsx** → Handles user login functionality with authentication.
• **Signup.jsx** → Provides registration interface for new users.
• **Dashboard.jsx** → Displays the main homepage showing all created grocery lists.
• **CreateList.jsx** → Allows users to create a new grocery list by entering list name and details.
• **ListDetails.jsx** → Displays all items inside a selected grocery list with options to add, update, delete, and mark items as purchased.
• **AddItem.jsx** → Provides form to add new grocery items including category, quantity, price, and image URL.
• **Profile.jsx** → Displays and manages user profile details.

COMPONENTS (REUSABLE PARTS)

• **Navbar.jsx** → Provides navigation across different pages such as dashboard and profile.
• **Footer.jsx** → Displays common footer section across all pages.
• **Home.jsx** → Acts as the landing page of the application.
• **UI Components (ui/)** → Contains reusable elements like buttons, forms, and modals to maintain consistency and reduce code duplication.

FEATURES

• Responsive User Interface:

The application is designed using Tailwind CSS to ensure a responsive and consistent interface across devices such as mobile, tablet, and desktop.

• Component-Based Architecture:

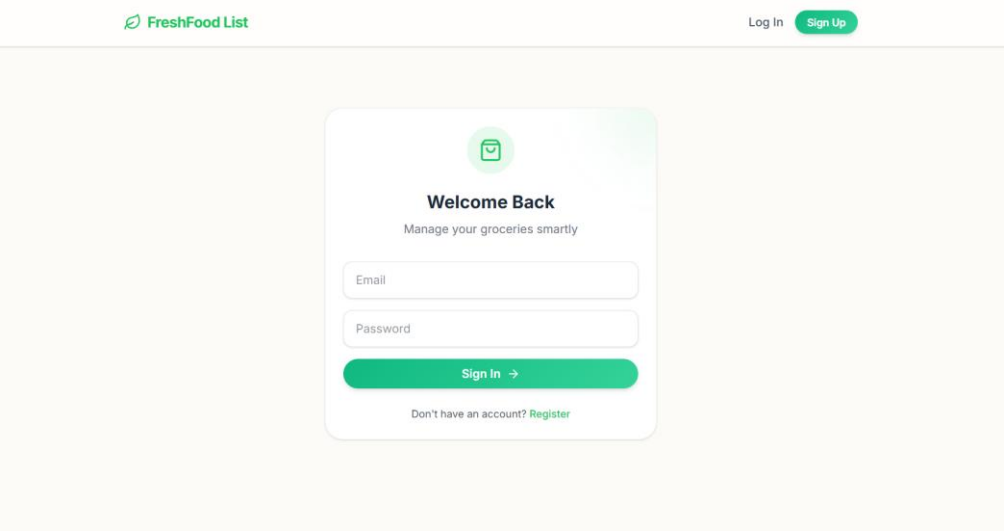
The frontend is built using a modular React approach, allowing reusable components and improving maintainability and scalability.

• State Management:

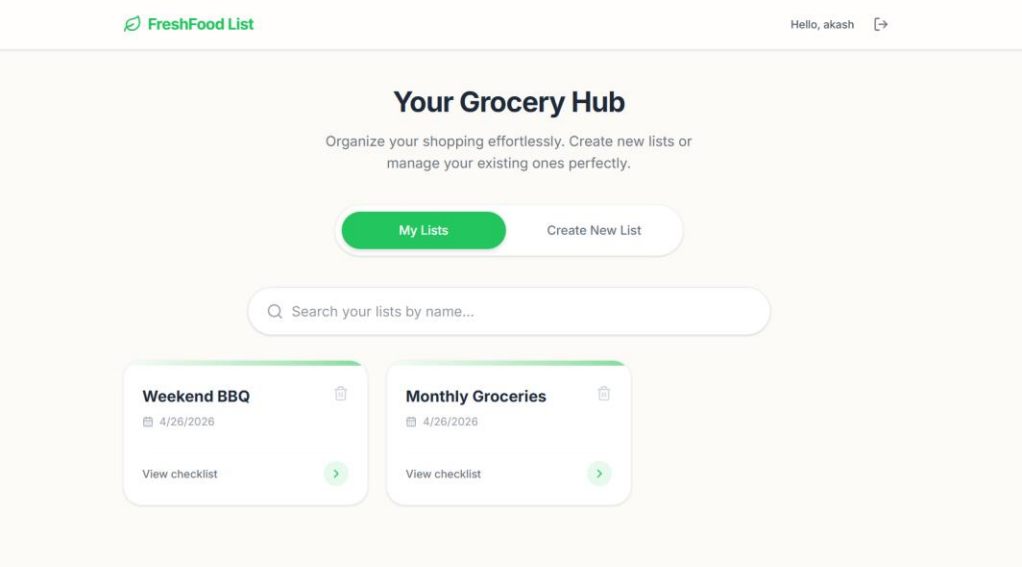
React Hooks are used for managing state efficiently, enabling dynamic updates and smooth interaction between components.

Output Screen Shots:

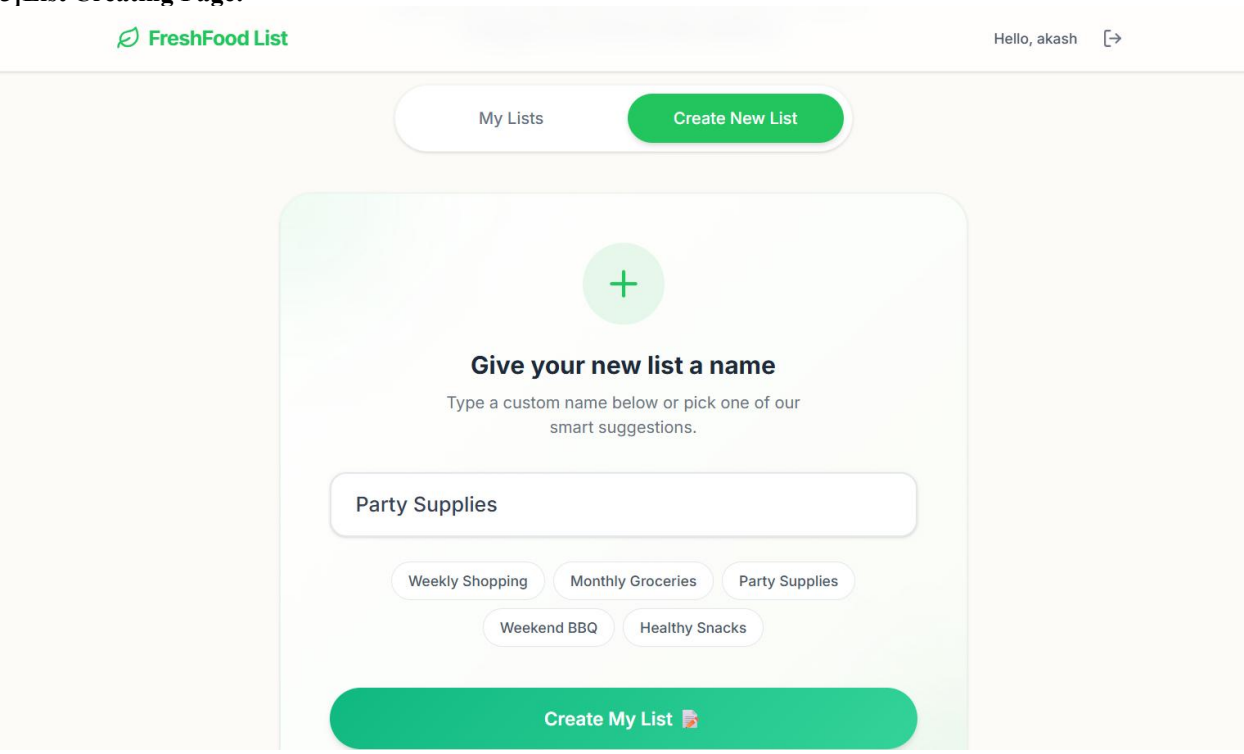
1]Login page:



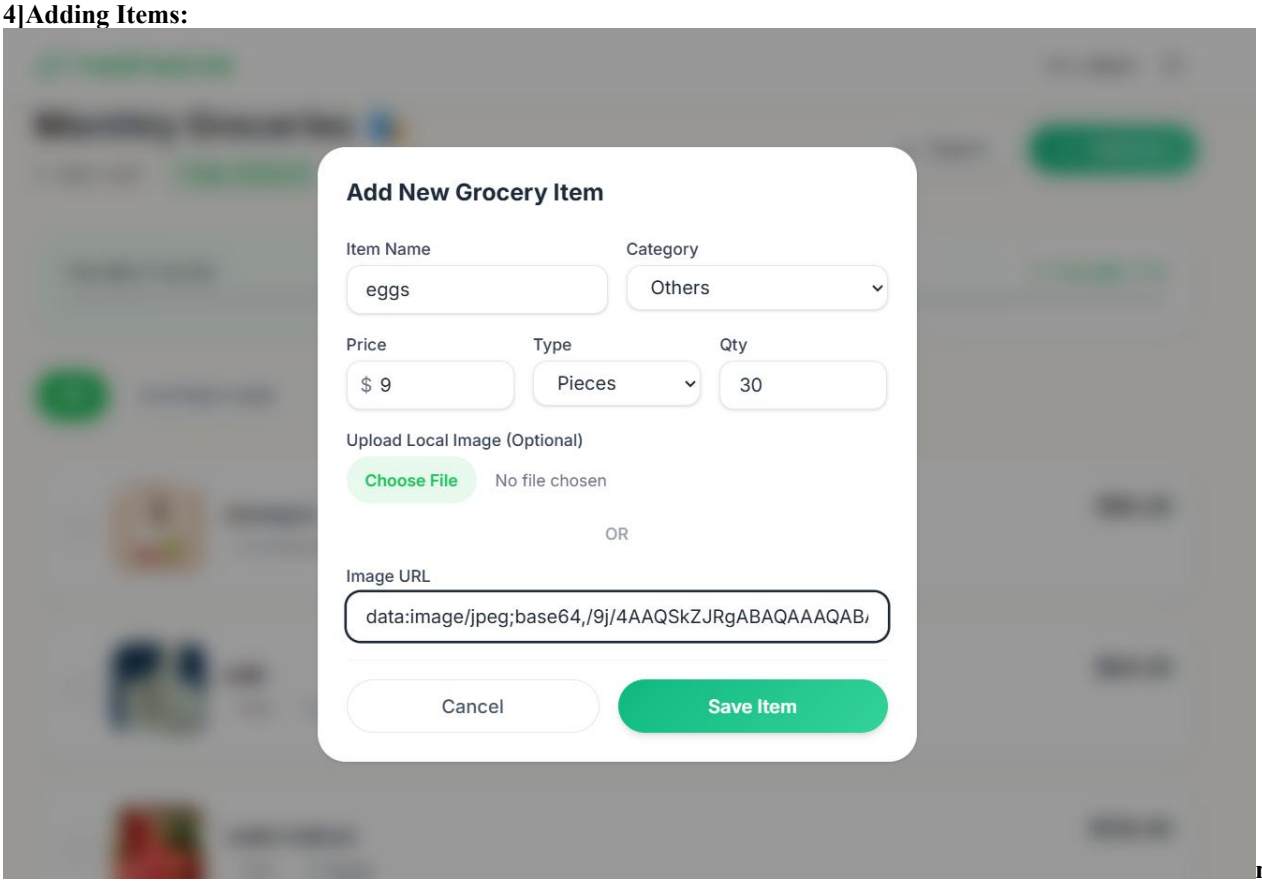
2]Home page:



3]List Creating Page:

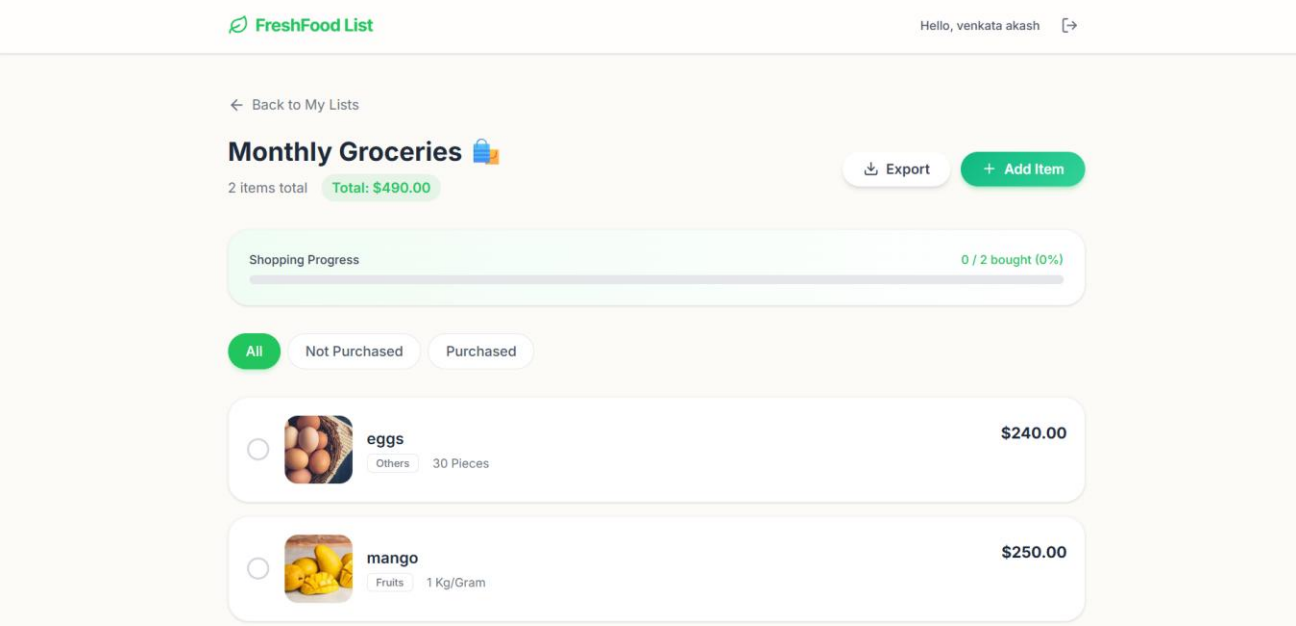


4]Adding Items:



m

5]List Details Page:



PROJECT OVERVIEW AND CODE EXPLANATION VIDEOS:

Frontend Explanation video:

https://drive.google.com/file/d/1634agsgQEUa8GjnCsEq6FJwhhIKgvAKl/view?usp=drive_link

Output Explanation Video:

https://drive.google.com/file/d/1-AyHh5gLuXaKqoYULaxuiNLpQ2pTPLJu/view?usp=drive_link

GITHUB LINK:

https://github.com/AP24110010877/full-stack-project-